ASSignment:-3

Name: Kaushal kumar Ray
PRN: 240205241023
Branch: B.Tech CSE with AI & ML
Semester: IV
Subject: operating system

Q1) Explain the critical section problem in detail. Discuss all three requirements: mutual exclusion, Progress and Bounded waiting.

→ The critical section problem deals with controlling access to shared resources so that only one process executes its critical section at a time, avoiding data inconsistency.

- Mutual Exclusion:
  only one process can enter the critical section at a time.
  Ex: updating a shared variable.

- Progress:
  If no process is inside, the next process is selected without dealay.

- Bounded waiting:
  A process will not wait indefinitely; there is a limit on waiting time.

② Discuss Peterson's Algorithm in detail. Explain how it ensures mutual exclusion.

→ A software solution for two processes to achieve mutual exclusion using shared variables.

key idea:

- flag [i] → interest
- turn → priority

Pseudocode

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j);

/* critical section */

flag[i] = false;
```

works:

- Ensures mutual exclusion
- Provides fairness (bounded waiting)

③ Explain the producer-consumer problem using semaphores with proper pseudocode.

→ A synchronization problem where producers generate data and consumers use it via a shared buffer.

Seamaphores
- mutex = 1
- empty = n
- full = 0

Producer:

```
wait (empty);
wait (mutex);
produce item;
signal (mutex);
signal (full);
```

consumer:

```
wait (full);
wait (mutex);
consume item;
signal (mutex);
signal (empty);
```

(4) Discuss different types of semaphores and explain how they solve synchronization problems.

→ A semaphore is an integer variable used for signaling and managing concurrent access to shared resources

- Binary semaphore: (value 0 or 1).
- used for mutual Exclusion (mutex). Ensures only one process enters a critical section.

- Counting semaphore (value 0 to N).
  used for Resource management. Tracks available instances of a resource.

To solve synchronization problem

1) critical section: Initialize semaphore to 1. Process calls wait(), executes, then call signal().

ii) Synchronization (ordering): To make Task B run after Task A, initialize semaphore to 0. Task A signals upon completion, Task B waits for that signal.

iii) Producer consumer: uses empty and full semaphores to manage buffer space and item availability.

(5) Explain Reader-writer problem with solution using semaphores and discuss starvation.

→ The Reader-writer problem is a synchronization problem where multiple processes can read shared data simultaneously, but only one process can write at a time.

Conditions:

- multiple readers allowed, writers require exclusive access

Solution using semaphores

- variables
  - mutex → protects read count
  - wrt → controls access to resources

Reader:

```
wait (mutex);
read_count ++;
if (read_count == 1)
    wait (wrt);
signal (mutex);

/* Reading */
```

```
wait (mutex);
  read_count --;
  if (read_count == 0)
    signal (wrt);
  signal (mutex);
```

writer:

```
wait (wrt);
  /* writing */

signal (wrt);
```

* Starvation
- It occurs when a process waits indefinitely.

- In this problem, writers may starve if readers continuously arrive

- Solution: give priority to writers.

⑥ Describe the Dining philosophers problem and explain different approaches to solve it.

→ The Dining philosophers Problem is a classical synchronization problem that illustrates issues like deadlock and starvation when multiple processes complete for limited resources.

Problem Description
- 5 philosophers sit around a table.
- Each needs two chopsticks to eat.
- Chopsticks are shared resources

Issues
- Deadlock
- starvation

Solutions
1) Resource ordering
11) Allow only 4 philosophers
111) semaphore solution
1V) Monitor-based solution

7 Explain Deadlock in detail, including definition, Necessary conditions, prevention techniques.

→ Deadlock is a situation in which a set of processes are blocked because each process is holding a resource and waiting for another resource held by another process.

Necessary conditions (Coffman conditions)

i) mutual exclusion
ii) Hold and wait
iii) No Preemption
iv) circular wait

prevention Techniques

- Eliminate one of the above conditions.
- Example : enforce resource ordering.

(8) Discuss Deadlock Avoidance and Banker's Algorithm with example:

→ Deadlock avoidance is a technique where the system ensures that it never enters an unsafe state by carefully allocating resources.

Banker's Algorithm

- It checks whether granting a request keeps the system in a safe state

Steps

1) check request ≤ need
2) Check request ≤ available
3) Temporarily allocate
4) Check safe sequence.

Example:

- If resources are sufficient and safe sequence exists → allocation allowed.

(9) Explain Deadlock Detection and Recovery Mechanisms in detail.

→ Deadlock detection identifies whether a deadlock has occurred and recovery methods are used to resolve it.

Detection:

- The system checks if processes are stuck waiting forever
- Uses a wait-for Graph
- Each node = Process, edge = waiting for another process
- If there's a cycle → deadlock exists.

For multiple resource instance, more complex algo. Banker's algo are used.

Recovery:

Once detected, the system break the deadlock:

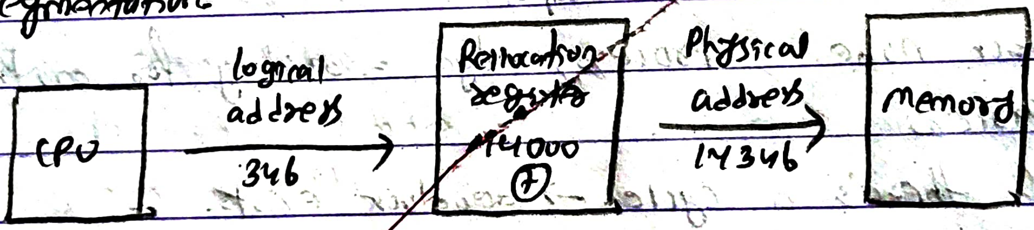
1) Process Termination
- Kill all deadlocked processes, or
- Kill one-by-one until deadlock is removed

2) Resource Preemption
- Forcefully take resources from some processes
- Reassign them to others to resolve the cycle

(10) Describe memory management techniques and compare contiguous and non-contiguous allocation.

→ Memory management is the process of managing computer memory by allocating and deallocating space to processes efficiently.

Techniques
- contiguous allocation
- Paging
- segmentation



comparison:

Feature	contiguous	Non-contiguous
- Allocation	continuous	scattered
- fragmentation	External	Reduced
- flexibility	Low	High

Memory is center to the operation of modern computer system. Memory contains of a large area of words and bytes, each with an average.

(11) Explain paging in detail with logical and physical address mapping.

→ Pagging is a memory management technique in which the logical address space of a process is divided into fixed-size blocks called pages, and physical memory is divided into equal-size blocks called frames. A page table is used to map pages to frames for execution

Concept:-
- Pages and frames are of equal size.
- Each process has its own page table
- Eliminates external fragmentation

Address mapping
- Logical address consists of:
  - Page Number (P)
  - ~~offset~~ offset (d)

Mapping Process:
1) CPU generates logical address (P,d)
2) Page table gives frame number (f)
3) Physical address = (f,d)

Formula:-
Physical Address = Frame Number × Page size + offset

(12) Explain Segmentation in detail and compare it with Paging.

→ Segmentation is a memory management technique in which a program is divided into variable-sized logical units called segments (like code, data, stack). Each segment represents a meaningful part of the program.

Basic concept:

- Memory is divided based on logical structure, not fixed size
- Each segment has:
  - ↳ Segment number (s)
  - ↳ Offset (d)
- A segment table is used for mapping

Each entry in segment table contains:

- Base → starting address of segment in memory
- Limit → length (size) of segment

* Logical to physical address mapping

1) CPU generates logical address (segment number, offset)
2) Segment number is used to access segment table
3) Check: offset < Limit (for protection)
4) Physical address = Base + offset

Formula :

Physical Address = Base Address + offset

Comparission :

Feature	segmentation	Paging
Division	logical (code, data, stack)	fixed-size pages
size	variable	fixed
Address format	segment + offset	page + offset
fragmentation	External	Internal
User visibility	visible to programmer	Invisible
Table used	segment Table	Page table
memory allocation	complex	simple

24-4-2026